Document Version: 1.0
Runner Version: 1.0.0
Test Version: 1.0.0

This document summarizes the public Test API declared in `include/testapi.h`. It focuses on the helper-specific return conventions and the option structs used by the comparison helpers.

Preface

This document is intended for BriteTest users and contributors who need quick access to the definitions of terms used in BriteTest or to browse through the terms.

For a list of other BriteTest documents and the BriteTest repository layout, see the Documentation Guide.

For a glossary of terms, see the Glossary Reference.

Document Version History

Document	Runner	Test	Date	Comment	Author/Editor
1.0	1.0.0	1.0.0	2026-06-11	Initial version.	Paul Sinclair

- The **Document** column records the document's version with the format `M.u` (Major, update).
- The **Runner** column records the BriteTest Runner API version current at the time this document version was published and is defined by its `RA_RUNNER_VERSION` macro.
- The **Test** column records the BriteTest Test API version current at the time this document version was published and is defined by its `RA_TEST_VERSION` macro.
- Both Runner and Test use the version format `"M.m.p"` (Major, minor, patch).
- `M` is the same for the Document, Runner, and Test versions.

The document's update version tracks released updates to this document and does not correspond to a minor or patch version. `u` increments when document changes are published in a release without a change to `M`, and it resets to `0` when `M` is incremented.

1. Return Conventions

1.1 Return Styles

BriteTest test helpers use three return styles:

- Structured helpers return `RA_OK`, `RA_MISMATCH`, `RA_TIMEOUT`, or an `RA_E*` error code.
- Predicate helpers return `1` for true or match, `0` for false or no match, and `-1` on invalid input or runtime error.
- Process helpers that model command completion return `0` for success, `1` for timeout, and `-1` for invalid input or setup failure.

1.2 Assertion Guidance

Use the helper family to decide what to check:

- File, JSON, metadata, and normalized-text comparisons should usually be checked against `RA_OK` or `RA_MISMATCH`.
- Existence, string, wildcard, and regex predicates should usually be checked against `1` or `0`.
- Command-execution helpers should usually be checked against `0` or `1`.

2. Comparison Option Structs

The comparison helpers use small option structs instead of mixed boolean and flag parameter lists.

2.1 ra_path_compare_options_t

Used by `ta_compare_paths_with_options`.

```
typedef struct {
    int flags;
} ra_path_compare_options_t;
```

Supported flags:

- `RA_FILECMP_IGNORE_TRAILING_WHITESPACE`
- `RA_FILECMP_IGNORE_EMPTY_LINES`
- `RA_FILECMP_IGNORE_TIMESTAMPS`
- `RA_FILECMP_IGNORE_LINE_ENDINGS`
- `RA_FILECMP_CASE_INSENSITIVE`

Default initializer:

```
#define RA_PATH_COMPARE_OPTIONS_INIT {0}
```

2.2 ra_path_metadata_compare_options_t

Used by `ta_compare_path_metadata`.

```
typedef struct {
    int flags;
} ra_path_metadata_compare_options_t;
```

Supported flags:

- `RA_STATCMP_SIZE`
- `RA_STATCMP_PERMS`
- `RA_STATCMP_MTIME`
- `RA_STATCMP_OWNER`
- `RA_STATCMP_TYPE`

Default initializer:

```
#define RA_PATH_METADATA_COMPARE_OPTIONS_INIT {0}
```

Passing `NULL` or the default initializer uses the common default metadata set: size, permissions, and type.

2.3 ra_json_compare_options_t

Used by `ta_compare_json_with_limit`.

```
typedef struct {
    size_t max_bytes;
```

```
    int ignore_key_order;
} ra_json_compare_options_t;
```

Default initializer:

```
#define RA_JSON_COMPARE_OPTIONS_INIT {0, 0}
```

Use `ignore_key_order` when semantic JSON equality matters more than source ordering. Use `max_bytes` when the caller wants a tighter bound than the implementation default.

2.4 `ra_text_compare_options_t`

Used by `ta_compare_text_normalized`.

```
typedef struct {
    int ignore_whitespace;
    int ignore_line_endings;
} ra_text_compare_options_t;
```

Default initializer:

```
#define RA_TEXT_COMPARE_OPTIONS_INIT {0, 0}
```

3. Common Usage Patterns

3.1 Compare Two Paths While Ignoring Timestamps

```
const ra_path_compare_options_t options = {
    RA_FILECMP_IGNORE_TIMESTAMPS
};

RA_ASSERT(ta_compare_paths_with_options(expected_path, actual_path, &options) == RA_OK,
```

3.2 Compare Path Metadata Including Modification Time

```
const ra_path_metadata_compare_options_t options = {
    RA_STATCMP_SIZE | RA_STATCMP_PERMS | RA_STATCMP_TYPE | RA_STATCMP_MTIME
};

RA_ASSERT(ta_compare_path_metadata(left_path, right_path, &options) == RA_OK, 0);
```

3.3 Compare JSON While Ignoring Object Key Order

```
const ra_json_compare_options_t options = {
    0,
    1
};

RA_ASSERT(ta_compare_json_with_limit(expected_json, actual_json, &options) == RA_OK, 0);
```

3.4 Compare Text While Ignoring Whitespace and Line Endings

```
const ra_text_compare_options_t options = {
    1,
    1
};

RA_ASSERT(ta_compare_text_normalized(left_text, right_text, &options) == RA_OK, 0);
```

4. Convenience Forms

- `ta_compare_paths` performs a byte-for-byte path comparison.
- `ta_compare_json` performs a strict JSON comparison using default limits.
- `ta_compare_file_lines` compares files line-by-line and returns `RA_OK` or `RA_MISMATCH`.

For the full declaration list and per-function comments, see `include/testapi.h`.